CLAUDE BLOG / APR 2026 / CHINESE EDITION

多智能体协同模式

五种模式、各自代价，以及什么时候该从一种模式演进到下一种。

Generator-Verifier | Orchestrator-Subagent | Agent Teams | Message Bus | Shared State

不要从“更高级”开始，而要从“最简单能跑”开始

Q1

验证标准明确吗？

如果输出好坏可以被清楚检查，先考虑生成器-验证器。

Q2

任务能清楚拆分吗？

如果子任务边界清楚且依赖少，协调者-子代理通常最省心。

Q3

工作要长期并行吗？

如果 worker 需要长期保留上下文，再看 agent teams。

Q4

信息靠事件流动还是靠知识积累？

前者偏 message bus，后者偏 shared state。

文章的核心建议：先用最简单的模式，看它卡在哪里，再按真实瓶颈演进，而不是为“架构感”过度设计。

模式 1：生成器-验证器适合“质量关键且可检查”的输出

HOW IT WORKS

生成初稿 → 验证标准 → 反馈重试

- 生成器先产出初始结果
- 验证器按明确标准检查是否通过
- 不过关就带着具体反馈回到下一轮

适合场景

代码生成配测试、事实核查、合规审查、评分 rubric。

主要风险

验证标准模糊会变成“形式上的质控”；循环太久还会不收敛。

关键不是有没有 verifier，而是 **verification 到底具体意味着什么**。

模式 2：协调者-子代理适合“边界清楚、输出明确”的任务拆分

MENTAL MODEL

一个 lead agent 负责规划、分派、汇总；子代理只处理自己那一小块，并把蒸馏结果带回来。

Orchestrator

安全检查

测试覆盖

风格一致性

为什么常用

- 主代理保留全局目标
- 子代理上下文更聚焦
- 很适合代码审查、资料搜集、独立检查项

卡点

- 信息必须回到 orchestrator 再转发，容易丢细节
- 不做并行时，成本上去了，速度却没提升

模式 3：Agent Teams 适合“可独立并行、且需要持续上下文”的长期工作

核心差异

Worker 是持续存在的

不是做完一次就销毁，而是跨多次任务保留熟悉度与领域记忆。

典型例子

大规模代码迁移

每个 teammate 负责一个服务，连续处理依赖、代码、测试和验证。

隐性成本

冲突与完成检测更难

不同 worker 完成时间不同，共享代码库时还可能互相踩文件。

判断点：如果子任务需要“长期熟悉自己的那块领域”，一次性 subagent 往往不够，team 模式才会开始显出价值。

模式 4：Message Bus 适合“事件驱动、生态会不断长大”的系统

PUBLISH / SUBSCRIBE

告警进入	Router / Topic	网络调查
身份分析	上下文补充	响应协调

代理不再彼此硬编码连接，而是围绕 topic 订阅与发布事件。

适合场景

安全运营、异步工作流、能力会持续新增的 agent pipeline。

主要风险

路由错误常常是“静默失败”；调试时需要很强的日志和关联追踪。

何时值得上

当 orchestrator 里塞满条件分支，且新事件类型越来越多时。

模式 5：Shared State 适合“彼此研究成果要实时互相影响”的协作探索

HOW IT WORKS

- 没有中心协调者，所有代理直接读写共享存储
- 一个代理的新发现，会马上变成另一个代理的输入
- 共享库会逐步长成一个不断更新的知识底座

WHAT CAN GO WRONG

- 重复研究同一条线索
- 互相触发的 reactive loops 持续烧 token
- 没有终止条件时，很难判断什么时候“已经足够好”

论文 | 行业 | 专利 | 新闻 | 共享知识库

如何选择：四组最容易混淆的分界线

协调者-子代理 VS AGENT TEAMS	看 worker 需不需要跨多轮保留上下文。
协调者-子代理 VS MESSAGE BUS	看流程是预先已知，还是由事件动态长出来。
AGENT TEAMS VS SHARED STATE	看成员之间是否需要实时共享中间发现。
MESSAGE BUS VS SHARED STATE	看系统是在传递离散事件，还是在共同累积知识库。

> 文章背后的统一原则是 **context-centric decomposition**：按每个 agent 需要什么上下文来拆，不要只按职能名字来拆。

推荐的演进路径：从一个可控核心出发，再按瓶颈加复杂度

1

先从 **Orchestrator-Subagent** 开始，覆盖大多数问题。

2

如果质量风险高，再局部加 **Generator-Verifier**。

3

如果 worker 需要长期自治，就演进成 **Agent Teams**。

4

如果系统靠事件扩张或知识共建，再分别转向 **Message Bus** 或 **Shared State**。

常见混搭 A

整体流程用 orchestrator，某个协作密集子任务内部用 shared state。

常见混搭 B

事件路由用 message bus，每类事件后面接 team-style workers 持续处理。

最后一页：把模式当作积木，不要把它们当作信仰

GENERATOR-VERIFIER	ORCHESTRATOR-SUBAGENT	AGENT TEAMS	MESSAGE BUS	SHARED STATE
质量关键	清晰拆分	长期并行	事件驱动	协作发现

默认起点不是“最酷的架构”，而是 **最简单、最可观测、最容易验证的那一个**。真正的升级，应该由瓶颈触发，而不是由想象触发。